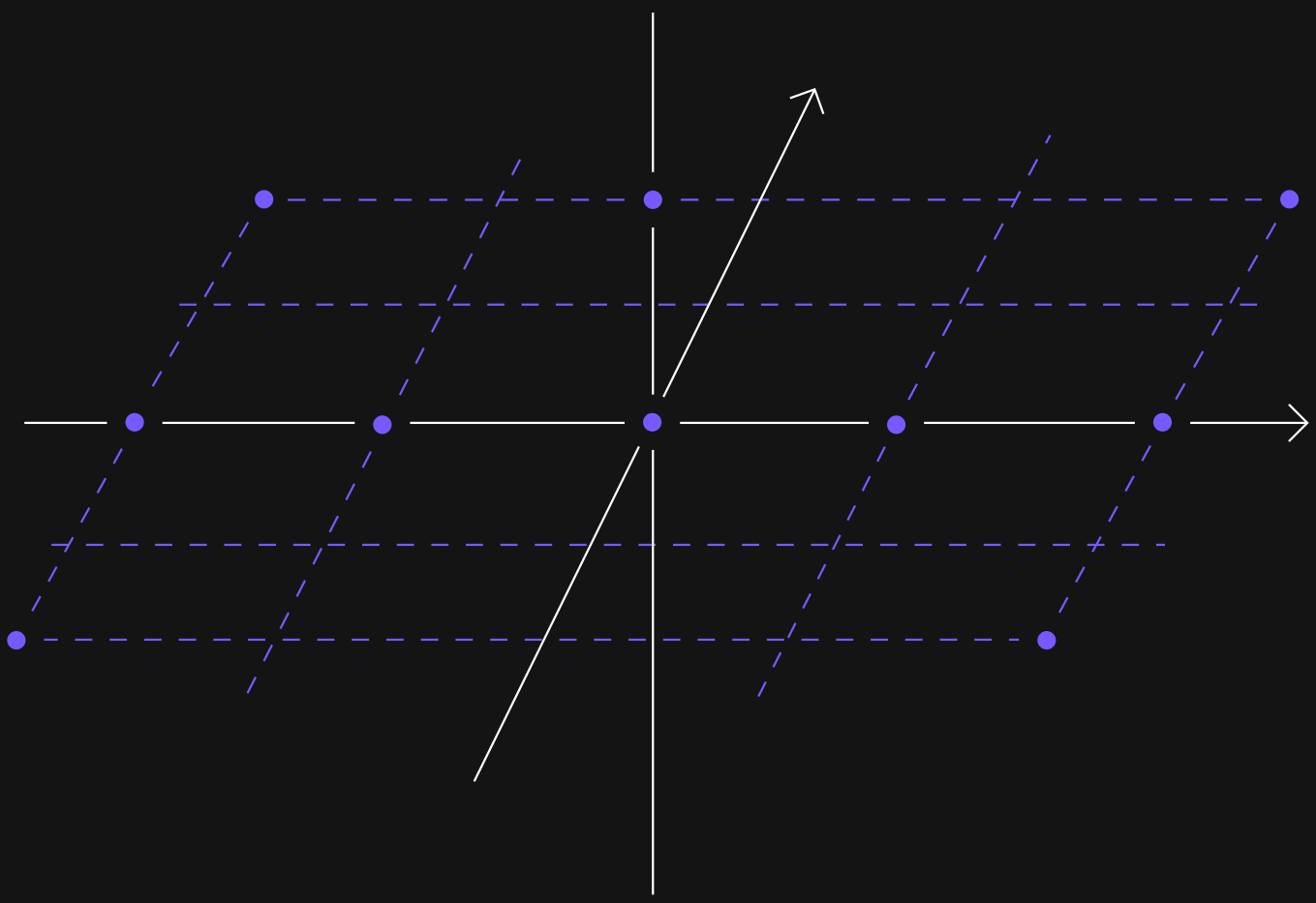
Разработка

Лонгрид

Сетевые технологии

02

Стек протоколов. Уровень представления



СЕТЕВЫЕ ТЕХНОЛОГИИ · ЛЕКЦИЯ 2 · ЛОНГРИД

Представление данных, TLS и Web-протоколы

Как интерпретировать содержимое сообщений: MIME, кодировки и схемы данных. Как TLS и сертификаты защищают обмен.

Материалы курса · запуск 2026

Содержание

01 Данные доставлены. Что находится внутри?

02 MIME: представление содержимого письма

03 Те же задачи в HTTP

04 Полнота сообщения и порядок обработки

05 Схемы, сериализация и ASN.1

06 Защита соединения и инфраструктура доверия

07 TLS: согласование и защищённый обмен

08 HTTP/1.1, HTTP/2 и HTTP/3

09 Самопроверка и диагностический отчёт

10 Дополнительные источники

1. Данные доставлены. Что находится внутри?

Где мы в модели OSI

В первой лекции мы начали с прикладного уровня, L7. Теперь спускаемся на один шаг к уровню представления, L6. Это второй уровень сверху, а не канальный L2.

Уровень	Название	Положение в курсе
L7	Прикладной	Первая лекция: HTTP и другие прикладные диалоги
L6	Представления	Сегодня: формат данных и защита обмена
L5	Сеансовый	Ниже по модели
L4	Транспортный	Ниже по модели
L3	Сетевой	Ниже по модели
L2	Канальный	Ниже по модели
L1	Физический	Ниже по модели

OSI разделяет функции сетевого взаимодействия. Реальный протокол может выполнять задачи нескольких уровней. В TCP/IP представлением данных часто занимаются прикладные протоколы, библиотеки и форматы, а не отдельный универсальный «протокол L6». Защита тоже возможна на разных уровнях.

Общий вопрос после прикладного обмена

Нашим основным примером был HTTP: клиент отправил запрос, сервер вернул ответ. Теперь посмотрим внутрь тела. Даже при статусе 200 нужно понять, пришёл текст или бинарный файл, как читать символы и применялось ли сжатие. Чтобы разобрать ошибку, потребуется различить состояние прикладного обмена, полноту сообщения и интерпретацию содержимого.

Та же задача возникает в других протоколах, с которыми мы познакомились в конце первой лекции. SMTP передаёт письма между участниками почтовой системы. IMAP и POP3 дают клиенту доступ к сообщениям в ящике. Получатель должен разобраться в содержимом письма так же, как HTTP-клиент должен разобраться в теле ответа.

Поэтому общий вопрос звучит так: данные доставлены, но что находится внутри и по каким правилам их интерпретировать?

Какие механизмы будем рассматривать

Задача	Примеры механизмов
Представить символы в байтах	Кодировки, например UTF-8
Описать тип содержимого и части	MIME
Передать структуру данных	JSON, Protobuf, ASN.1
Сократить объём содержимого	Сжатие, например gzip
Защитить обмен и проверить сторону	TLS и инфраструктура доверия PKI

Здесь перечислены разные сущности: форматы, стандарты представления и протоколы. MIME является одним из механизмов этой темы, а не заменяет её целиком. Общие задачи встречаются в почте, HTTP, RPC, сетевом управлении и других системах.

Почему подробный разбор начнём с почты

Электронное письмо особенно наглядно: в одном сообщении могут одновременно находиться русский текст, HTML-версия, встроенная картинка и файл отчёта. На одном знакомом объекте видны сразу несколько задач представления.

MIME исторически появился именно в почте. Поэтому почтовый пример поможет понять и устройство механизма, и причины его возникновения. После разбора типов, частей и кодирования вернёмся к HTTP и посмотрим, как он использует те же идеи.

Дальнейший порядок лекции:

1. MIME и составные сообщения на примере электронной почты.
2. Повторное использование общих механизмов представления в HTTP.
3. Схемы, сериализация и ASN.1 за пределами Web.
4. Защита соединения, TLS и инфраструктура доверия PKI.
5. Общее и различия HTTP/1.1, HTTP/2 и HTTP/3.

После занятия важно уметь прочитать метаданные сообщения, восстановить порядок обработки тела, объяснить основные проверки сертификата и предложить следующую диагностическую проверку. Запоминать каталоги всех типов ASN.1 или писать криптографические алгоритмы не требуется.

Как читать примеры

Примеры с пометкой «фрагмент» намеренно показывают только нужную часть обмена. Они не заменяют полное корректное сообщение. В почтовых и HTTP/1.1-примерах конец строки на проводе обозначается двумя байтами CRLF: шестнадцатеричными 0D 0A. В отображении ниже это обычный перенос строки.

Учебные домены example.org, example.net и course.test не обозначают доступные учебные сервисы. Команды, которые используют course.test, требуют отдельно подготовленного сервера и сертификатов. Локальные примеры на Python работают без этого сервера.

2. MIME: представление содержимого письма

Почему возник MIME

Исторический формат интернет-почты ориентировался на строки US-ASCII. Это создавало ограничения для международного текста, бинарных вложений и сложных документов. Требовалось расширить содержимое письма так, чтобы оно проходило через существующую инфраструктуру.

MIME, Multipurpose Internet Mail Extensions, описывает типы содержимого, его кодирование и структуру составного сообщения. Команды SMTP при этом продолжают отвечать за передачу письма. MIME объясняет почтовому клиенту, как интерпретировать полученное содержимое.

Нельзя переносить историческое ограничение на всю современную почту. Расширения 8BITMIME, BINARYMIME и SMTPUTF8 позволяют дополнительные формы передачи. Их использование зависит от согласованных возможностей участников. Они не отменяют необходимости описывать структуру письма и типы частей.

Заголовки и тело

Сообщение содержит заголовки, затем пустую строку, затем тело. В MIME некоторые заголовки описывают именно содержимое:

Поле	Назначение
MIME-Version	Версия MIME. Для рассматриваемого формата это 1.0
Content-Type	Media type содержимого и его параметры
Content-Transfer-Encoding	Применённое кодирование тела или ограничения на его байтовый вид
Content-Disposition	Подсказка о представлении содержимого, например attachment
Content-ID	Идентификатор части, на которую можно сослаться

Учебное сокращение письма:

```
From: sender@example.org
To: student@example.net
Subject: Course example
MIME-Version: 1.0
Content-Type: text/plain; charset=UTF-8
Content-Transfer-Encoding: base64

0J/RgNC40LLQtdGC
```

Здесь тело ещё не является непосредственно текстом письма. Сначала Base64-декодер восстанавливает байты, затем UTF-8-декодер превращает их в строку «Привет». Служебные поля полного письма, такие как Date и Message-ID, в этом учебном примере опущены.

Media type

Media type имеет вид type/subtype. Он задаёт способ интерпретации содержимого. Параметры после точки с запятой уточняют представление.

Тип	Примеры	Как понимать
text	text/plain, text/html, text/css	Текст определённого формата
image	image/png, image/jpeg, image/gif	Изображение
audio	audio/mpeg, audio/ogg	Аудиоданные
video	video/mp4, video/mpeg	Видеоданные
application	application/json, application/pdf, application/zip	Данные конкретного формата или приложения
multipart	multipart/mixed, multipart/alternative	Составное содержимое
message	message/rfc822	Вложенное сообщение

application/octet-stream обозначает произвольные бинарные данные без более конкретного типа. Название файла не заменяет Content-Type. Расширение .json не доказывает, что файл содержит JSON, а объявленный тип не гарантирует корректность тела.

Параметр charset уточняет кодировку символов там, где он определён для соответствующего media type. Параметр boundary задаёт разделитель multipart. Исторически в Content-Type встречается name, но имя вложения обычно сообщают параметром filename поля Content-Disposition.

Правила параметров зависят от типа. Например, для application/json не определён параметр charset: межсистемный JSON обменивается в UTF-8 по правилам RFC 8259. Не следует автоматически применять настройки text/plain к любому содержимому.

Content-Disposition и Content-ID

Content-Disposition: attachment; filename="report.pdf" предлагает представить часть как отдельное вложение. Значение inline предлагает отображать её внутри документа, когда это поддерживается клиентом. Это подсказки представления, а не разрешение автоматически открывать исполняемый файл.

Имя файла приходит от другой стороны. Программа сохранения должна исключать выход из каталога, перезапись произвольных файлов и другие небезопасные операции с путями.

Для связанного ресурса можно задать Content-ID: plot@example.org. HTML-часть затем использует ссылку cid:plot@example.org. Адресоподобная запись является идентификатором части, а не указанием отправить письмо в такой ящик.

Multipart и границы частей

Multipart объединяет несколько частей. Каждая часть имеет собственные заголовки и тело. Объявление boundary задаёт строку, из которой строятся разделители:

```
Content-Type: multipart/mixed; boundary="part"

--part
Content-Type: text/plain; charset=US-ASCII

See attachment.
--part
Content-Type: text/plain; charset=US-ASCII
Content-Disposition: attachment; filename="report.txt"

OK
--part--
```

Это MIME-фрагмент. Значение boundary равно part, начало очередной части обозначается строкой --part, завершение составного тела обозначается --part--. Пустая строка отделяет заголовки каждой части от её содержимого. Разделитель выбирают так, чтобы его строка не совпала с содержимым частей.

Часть может сама содержать multipart. Поэтому сложное письмо имеет древовидную структуру. Нельзя надёжно разобрать произвольное письмо единственным разделением по пустой строке или фиксированному слову.

Подтип	Отношение частей	Типичный пример
mixed	Разные части общего сообщения	Текст и вложения
alternative	Альтернативные представления одного содержимого	Plain text и HTML
related	Взаимосвязанные ресурсы	HTML и картинки, на которые он ссылается
form-data	Именованные поля формы	Текстовое поле и файл в HTTP

Для alternative получатель выбирает подходящее поддерживаемое представление. По соглашению части располагают от менее к более предпочтительной. Это не набор независимых вложений, который обязательно следует показать целиком.

Для related клиент учитывает корневую часть и ссылки на связанные ресурсы. Multipart/mixed при необходимости может содержать multipart/alternative, а одна из альтернатив может быть multipart/related.

Content-Transfer-Encoding

Это поле нужно отличать от кодировки символов. Оно описывает преобразование тела для передачи либо ограничения на байтовый вид тела.

Значение	Смысл
7bit	Преобразования нет, содержимое соответствует ограничениям 7-битных строк
8bit	Преобразования нет, разрешены байты вне 7-битного диапазона при сохранении ограничений строк
binary	Преобразования нет, содержимое не ограничено моделью таких строк
quoted-printable	Некоторые байты записываются через знак = и две hex-цифры
base64	Произвольные байты переводятся в ограниченный печатный алфавит

Если поле отсутствует, MIME предполагает 7bit. Поэтому нельзя просто поместить UTF-8-кириллицу в пример письма и забыть о правилах передачи.

Quoted-printable позволяет оставить значительную часть ASCII-текста читаемой. Например, байт D0 можно представить как =D0. Для текста с большим количеством не-ASCII-байтов результат может быть длинным. Знак равенства в конце строки может обозначать мягкий перенос, который снимается при декодировании.

Base64 обрабатывает группы по три байта и выдаёт четыре символа. С учётом заполнения длина составляет 4 × ceil(n/3), без почтовых переносов строк. Для больших данных прибавка близка к одной трети. Base64 не уменьшает объём и не требует секретного ключа.

Самостоятельный разбор без почтового сервера

Предскажите результат программы до запуска:

```
import base64

encoded = b"0J/RgNC40LLQtdGC"
raw = base64.b64decode(encoded, validate=True)

print(raw.hex(" "))
print(raw.decode("utf-8"))
print(len(raw))
```

Ожидаемый результат: последовательность D0 9F D1 80 D0 B8 D0 B2 D0 B5 D1 82, строка «Привет» и длина 12. Здесь шесть букв занимают двенадцать байтов UTF-8.

Что будет, если вывести encoded как ASCII? Получится строка Base64, а не исходное слово. Что будет, если декодировать raw неверной кодировкой? Возможны искажение текста или ошибка декодирования. Доставка байтов при этом могла пройти без ошибок.

Источники: RFC 2045, RFC 2046, RFC 2183, RFC 2387, RFC 2392.

3. Те же задачи в HTTP

Что переиспользует HTTP

Media types и multipart полезны не только в почте. HTTP тоже должен сообщить получателю, что находится в теле и как это тело обрабатывать. При этом нельзя переносить правила каждого похожего заголовка из одного протокола в другой без проверки.

HTTP-ответ с несжатым текстом:

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=UTF-8
Content-Length: 12

Привет
```

У тела в этом примере нет завершающего перевода строки. Content-Length сообщает ровно 12 байтов, а не число букв и не размер всего ответа. Заголовки и пустая строка в эту длину не входят.

Статус 200 относится к результату HTTP-запроса. Он не гарантирует, что тело соответствует ожиданиям вызывающего кода. Программа, которая ждала JSON, не сможет использовать этот текст как JSON только потому, что статус успешный.

Три поля, которые легко перепутать

Поле	Контекст	Что описывает
Content-Transfer-Encoding	MIME в почте	Кодирование тела или его допустимый байтовый вид
Content-Encoding	HTTP	Преобразование представления, например gzip
Transfer-Encoding	HTTP/1.1	Способ передачи тела сообщения, например chunked

Charset описывает кодировку символов и решает отдельную задачу. Gzip не заменяет UTF-8, а Base64 не заменяет gzip.

Content-Encoding: gzip означает, что перед интерпретацией представления нужно снять gzip-кодирование. Если вместе с ним есть Content-Length, длина относится к кодированному, то есть сжатому содержимому. Важно учитывать, что HTTP-библиотеки могут автоматически распаковывать ответ. Тогда сохранённое библиотекой тело и Content-Length исходного сообщения могут относиться к разным стадиям обработки.

Согласование представления

Клиент может выразить предпочтения:

```
Accept: application/json
Accept-Encoding: gzip
```

Ответ сообщает фактически выбранный вариант:

```
Content-Type: application/json
Content-Encoding: gzip
```

Это фрагменты заголовков. Accept относится к media type, Accept-Encoding к допустимым кодированиям содержимого. Получатель должен читать фактический ответ, а не предполагать, что его предпочтение обязательно выполнено.

В зависимости от возможностей ресурса сервер может выбрать подходящее представление или ответить ошибкой, например 406. Если одно представление зависит от заголовков запроса, для корректного кэширования важен Vary. Подробности HTTP-кэшей не входят в это занятие.

Сжатие и текст

Gzip, deflate и Brotli применяются для сокращения объёма передаваемых данных. Brotli часто обозначается значением br. Для HTTP значение deflate имеет конкретный формат, включающий zlib-обёртку, а не произвольные «сырые» DEFLATE-байты.

Эффект сжатия зависит от содержимого. Уже сжатое изображение или архив может почти не уменьшиться. Нельзя без измерения утверждать, что один алгоритм всегда быстрее или компактнее другого.

Локальный пример показывает две разные операции:

```
import gzip

original = "Привет".encode("utf-8")
packed = gzip.compress(original, mtime=0)
restored = gzip.decompress(packed)

print("UTF-8 bytes:", len(original))
print("gzip bytes:", len(packed))
print(restored.decode("utf-8"))
assert restored == original
```

На таком коротком теле служебные данные gzip могут увеличить размер. Это нормальный результат, который помогает отделить назначение алгоритма от обещания уменьшить любой вход.

Multipart/form-data

Когда форма отправляет несколько полей и файл, HTTP может использовать multipart/form-data. Пример фрагмента:

```
Content-Type: multipart/form-data; boundary="form"

--form
Content-Disposition: form-data; name="student"

Alex
--form
Content-Disposition: form-data; name="file"; filename="a.txt"
Content-Type: text/plain

OK
--form--
```

Здесь name обозначает имя поля, filename имя файла, boundary разделитель частей. Полный запрос дополнительно содержит request-line, Host и корректное обозначение границ HTTP-тела.

HTTP может передавать бинарное содержимое без почтового Base64. Для multipart/form-data не нужно автоматически добавлять Content-Transfer-Encoding: base64 по аналогии с письмом. Практическая ошибка возникает, когда библиотека формирует тело с одним boundary, а программист вручную задаёт в Content-Type другое значение.

Источники: RFC 9110, представления, RFC 7578, RFC 8259.

4. Полнота сообщения и порядок обработки

Почему одного чтения недостаточно

Сетевой поток не обязан выдавать данные по границам HTTP-заголовков или прикладных сообщений. Один вызов read/recv может вернуть часть строки, несколько строк или лишь начало тела. Размер полученного фрагмента не обозначает конец сообщения.

Для HTTP/1.1 получатель определяет границу по правилам протокола. В упрощённой учебной модели:

- Content-Length задаёт число байтов тела.
- Transfer-Encoding: chunked задаёт последовательность чанков.

- Некоторые ответы имеют тело, ограниченное закрытием соединения.
- Некоторые ответы по определению не имеют тела.

Для настоящей реализации важен порядок правил, а не только наличие одного заголовка. Ответы на HEAD и ответы со статусами 1xx, 204 и 304 не содержат message body. Успешный CONNECT переводит соединение в режим туннеля. Запрос без указания длины или transfer coding не получает тело по правилу «читать до закрытия».

В ответе на HEAD или в допустимом ответе 304 поле Content-Length может характеризовать представление, которое было бы передано в другом обмене, а не требовать чтения тела текущего сообщения.

Получатель также должен учитывать конфликтующие поля. Одновременное присутствие Transfer-Encoding и Content-Length требует обработки по RFC и считается опасной неоднозначностью. В ограниченном учебном клиенте разумно явно отвергать такие сообщения, не пытаться угадывать намерение отправителя. Неодинаковое понимание границ посредниками связано с классом ошибок request smuggling.

Если ожидали 100 байтов по Content-Length, а соединение закрылось после 60, сообщение неполно. Если тело ограничено закрытием, корректное окончание и некоторые обрывы могут быть неразличимы только по признаку EOF. Тайм-аут сам по себе не является допустимым разделителем тела.

Chunked

Пример полного короткого ответа:

```
HTTP/1.1 200 OK
Content-Type: text/plain
Transfer-Encoding: chunked

4
Wiki
5
pedia
0
```

Размер каждого чанка записывается шестнадцатеричным числом. После строки размера идут соответствующие байты и CRLF. Последний чанк имеет нулевой размер. Затем могут идти trailer fields, а пустая строка завершает сообщение.

Результат снятия chunked в примере: Wikipedia, девять байтов. Строки 4, 5 и 0 не входят в восстановленное содержимое. Они служат для передачи тела сообщения.

Точное окончание примера без trailers: 0\r\n\r\n. Chunked допускает дополнительные элементы, включая расширения чанков. Полноценный парсер должен реализовать соответствующие правила либо явно определить поддерживаемый учебный профиль.

Конвейер обработки

Получатель сначала должен понимать границы HTTP-сообщения. Затем он снимает применённые преобразования и интерпретирует содержимое. Это логический порядок: реализация может обрабатывать поток постепенно, не накапливая целиком каждый промежуточный результат.

Этап	Пример действия	Пример сбоя
Границы HTTP	Прочитать Content-Length или разобрать chunked	Обрыв тела
Кодирование содержимого	Снять Content-Encoding: gzip	Ошибка распаковки
Структура multipart	Выделить части и их метаданные	Несовпадающий boundary
Текстовая ветвь	Преобразовать UTF-8 в строку	Неверные последовательности байтов
Бинарная ветвь	Передать байты декодеру изображения, PDF или DER	Некорректный формат
Синтаксис и схема	Разобрать JSON и проверить поля	Некорректный JSON или неожиданная структура

Таблица не требует выполнять каждый шаг для каждого сообщения. У обычного JSON нет стадии multipart. У бинарного изображения нет стадии декодирования текста через charset. Для частей составного сообщения соответствующая обработка выполняется отдельно.

Даже корректный JSON может содержать строку вместо ожидаемого числа, отсутствующее обязательное поле или значение вне допустимого диапазона. Синтаксическая корректность и соответствие модели приложения различаются.

Диагностический пример

Наблюдение: «клиент получил HTTP/1.1 200 OK, JSON-парсер сообщил ошибку».

Из этого недостаточно вывести, что сервер вернул полный JSON. Возможны разные ситуации:

Гипотеза	Различающая проверка
Пришёл HTML	Content-Type и начало фактического тела
Тело осталось сжатым	Content-Encoding и поведение библиотеки
Тело обрезано	Framing, ожидаемая длина, фактически полученные байты
Тело полное, но JSON неверный	Проверка синтаксиса полного представления
JSON корректен, не совпадает схема	Отделить парсинг от валидации модели

Потоковый парсер способен сообщить ошибку до получения последнего байта, поэтому сама ошибка парсера не подтверждает полноту сообщения.

При сборе материалов для коллег фиксируют время, URL, метод, статус, существенные поля, достигнутый этап и точную ошибку. Сохраняют данные на нужной стадии: «сырое тело на проводе» и «тело после автоматической распаковки» не всегда одно и то же. Перед передачей удаляют токены, cookies, персональные данные и другие секреты.

Источники: RFC 9112, длина тела, RFC 9112, chunked.

5. Схемы, сериализация и ASN.1

Текстовые и бинарные представления

Когда стороны обмениваются объектами, нужно договориться о типах полей, допустимых значениях и способе записи в байты. Внутренний объект Python или Java нельзя автоматически считать межсистемным форматом.

JSON описывает объекты, массивы и значения. XML описывает документы через элементы и атрибуты. Для обоих можно отдельно задать схему. Protocol Buffers использует объявления сообщений и полей в .proto, а также определённое бинарное кодирование. ASN.1 задаёт абстрактные типы и ограничения, к которым применяют выбранные encoding rules.

Вопрос	Почему важен
Нужна ли человеку читаемость?	Влияет на диагностику и работу с инструментами
Согласована ли схема?	Получатель должен знать типы и смысл полей
Как добавлять поля?	Старые и новые участники обновляются не одновременно
Насколько важен объём?	Цена байтов зависит от канала и частоты обмена
Требуется ли каноничность?	Для подписи важно точное байтовое представление

Бинарное кодирование не означает автоматически меньший размер и большую скорость в любых условиях. Результат зависит от данных, схемы, реализации и дополнительных преобразований, например сжатия.

При эволюции схемы важно различать отсутствие поля и значение по умолчанию, неизвестное поле и ошибку структуры. Стратегия совместимости должна быть частью протокола, а не случайным свойством парсера. Например, номера полей Protobuf имеют протокольное значение и не должны произвольно переиспользоваться.

Unicode и кодировки

Кодировка определяет, как текст записывается в байтах и как получатель восстанавливает его из байтов. Один и тот же текст может иметь разные байтовые представления.

Кодировка	Байты слова «Привет» в hex	Размер
UTF-8	D0 9F D1 80 D0 B8 D0 B2 D0 B5 D1 82	12 байтов
UTF-16 LE	1F 04 40 04 38 04 32 04 35 04 42 04	12 байтов
Windows-1251	CF F0 E8 E2 E5 F2	6 байтов
ASCII	Кириллицу представить нельзя	Нет представления

В строке UTF-16 выбран порядок little-endian (LE): младший байт идёт первым. Служебная метка порядка байтов, BOM, в пример не включена. Эти размеры относятся к конкретному слову и не являются правилом для всех символов.

Windows-1251, также называемая CP1251, представляет каждый поддерживаемый символ одним байтом. В ней есть кириллица, латиница и некоторые другие символы, но нет универсального набора для всех письменностей и эмодзи. Если прочитать байты Windows-1251 как UTF-8, можно получить ошибку декодирования; другие неверные сочетания дают искажённый текст.

Unicode определяет общий набор символов и их числовые обозначения. Например, букве «П» соответствует число 1055, которое обычно записывают как U+041F. Такое обозначение называют кодовой точкой. Это ещё не последовательность байтов: UTF-8 запишет «П» как D0 9F, а UTF-16 LE как 1F 04.

UTF-8 и UTF-16 являются способами кодирования Unicode, а не его подмножествами. Поэтому ставить Unicode в один ряд с UTF-8, UTF-16 и Windows-1251 как ещё одну альтернативную кодировку было бы смешением разных понятий.

UTF-8 сохраняет однобайтовую запись ASCII-диапазона. Символы вне него могут занимать несколько байтов. UTF-16 использует 16-битные кодовые единицы, а при записи в байты возникает вопрос порядка байтов. Некоторые символы требуют двух кодовых единиц.

Число байтов, кодовых точек и видимых символов может различаться. Например, видимый символ иногда состоит из нескольких кодовых точек с комбинирующими знаками. Для сетевого framing необходима длина в байтах, а не результат произвольной функции «длина строки».

ASCII остаётся важен для синтаксиса многих протоколов. Характеристика «устаревшая кодировка, про которую можно забыть» не помогает понять устройство HTTP/1.1 или SMTP.

ASN.1 как язык описания

ASN.1, Abstract Syntax Notation One, позволяет описать структуру значений независимо от языка программирования, выравнивания памяти и архитектуры компьютера.

```
Person ::= SEQUENCE {
  name  UTF8String,
  age   INTEGER (0..150),
  email UTF8String OPTIONAL
}
```

Здесь описан тип Person. SEQUENCE объединяет компоненты, age имеет ограниченный диапазон, email может отсутствовать. Это схема, а не готовый пакет сети. Чтобы получить байты, нужны конкретное значение и выбранные правила кодирования.

Краткий справочник типов:

Группа	Типы	Назначение
Простые значения	BOOLEAN, INTEGER, REAL, NULL, ENUMERATED	Логические, числовые и перечислимые значения
Последовательности битов и байтов	BIT STRING, OCTET STRING	Битовые поля и произвольные октеты
Идентификаторы	OBJECT IDENTIFIER	Обозначение объектов, типов и алгоритмов
Строки	UTF8String, NumericString, PrintableString	Текст с определёнными ограничениями
Время	UTCTime, GeneralizedTime	Формализованное представление времени
Составные типы	SEQUENCE, SET	Набор компонентов
Коллекции	SEQUENCE OF, SET OF	Несколько значений одного типа
Выбор	CHOICE	Один из объявленных вариантов

OPTIONAL допускает отсутствие компонента. DEFAULT задаёт значение по умолчанию по правилам схемы и кодирования. Эти механизмы помогают строить структурированный обмен, но не снимают необходимость согласовать смысл данных.

OID имеет иерархический вид, например 1.2.840.113549.1.1.11. Этот OID обозначает sha256WithRSAEncryption, алгоритм подписи. Несмотря на слово Encryption в историческом имени, в таком контексте речь идёт о подписи, а не о шифровании всего трафика.

BER, DER и TLV

BER, Basic Encoding Rules, допускает несколько способов кодирования некоторых значений. DER, Distinguished Encoding Rules, ограничивает BER для получения канонического представления. Для заданной структуры это помогает получить однозначные байты там, где их точное значение существенно, например при подписи.

В простом BER/DER-примере удобно видеть TLV:

```
02 01 2A
```

- 02: тег универсального типа INTEGER.
- 01: длина значения один байт.
- 2A: число 42.

Составной пример:

```
30 03 02 01 2A
```

Тег 30 обозначает constructed SEQUENCE. Её значение имеет длину три байта и содержит вложенный INTEGER.

В общем случае тег и длина могут занимать несколько байтов. BER также поддерживает неопределённую длину некоторых конструкций, а DER ограничивает допустимые варианты. Простой пример не является готовым алгоритмом разбора всех ASN.1-данных.

TLV встречается и вне ASN.1. Бинарный протокол с полями «тип, длина, значение» нельзя на этом основании записать в реализации BER или DER.

Другие правила кодирования

Правила	Идея
BER	Гибкое базовое кодирование
DER	Канонические ограничения BER
CER	Ещё один канонический набор правил для ASN.1
PER	Компактная упаковка с использованием информации из схемы
XER	Кодирование ASN.1-значений через XML
JER	Кодирование ASN.1-значений через JSON

PER, Packed Encoding Rules, особенно интересен там, где важен объём, например в протоколах радиодоступа. Схема и ограничения помогают не передавать часть избыточной информации. Это не обещание убрать все теги или служебные биты при любых условиях.

Где используется ASN.1

Область	Пример	Роль
Сертификаты	X.509	Структуры сертификатов, в рассматриваемом профиле DER
Запросы на сертификат	PKCS #10 CSR	Структура запроса и подписываемых данных
Управление сетью	SNMP	Сообщения и объекты, BER
Службы каталогов	LDAP	Сообщения с ограниченным набором BER
Мобильная сигнализация	MAP, CAP	Формальные структуры сообщений
Радиодоступ	RRC	ASN.1-структуры и PER

SNMP применяется для наблюдения и управления сетевыми устройствами. LDAP используется для доступа к каталогам. MAP и CAP связаны с сигнализацией и услугами мобильных сетей. RRC, Radio Resource Control, управляет радиосоединением между терминалом и сетью доступа.

В старом материале к примерам ASN.1 также относились RADIUS, Diameter и NAS. Эту классификацию не сохраняем: RADIUS имеет свой формат атрибутов, Diameter свои AVP, а форматы NAS LTE/5G определяются соответствующими спецификациями сообщений и информационных элементов. Наличие бинарных структур или TLV не означает ASN.1. Это уточнение классификации, а не исключение технологий из курса.

Источники: ITU-T X.680, X.690, X.691, X.693, X.697, SNMP, RFC 3417, LDAP, RFC 4511, Protocol Buffers.

6. Защита соединения и инфраструктура доверия

Какие проблемы решает защита

Понятный формат сообщения не делает канал безопасным. Другой участник сети может наблюдать трафик, пытаться изменять его или выдавать себя за нужный сервис.

Конфиденциальность ограничивает чтение содержимого посторонними. Целостность позволяет обнаружить несанкционированное изменение защищённых данных. Аутентификация подтверждает сторону в рамках выбранной модели идентичности.

TLS, Transport Layer Security, решает эти задачи для соединения. Обычное HTTPS-соединение использует сертификат сервера для его аутентификации. Прикладная авторизация пользователя остаётся отдельной задачей. Возможна и аутентификация клиента сертификатом, mTLS (mutual TLS), но она не обязательна для обычного HTTPS.

Защита бывает и на других уровнях: например, IPsec и MACsec. Поэтому нельзя утверждать, что уровни ниже представления по определению не обеспечивают безопасность. На этой лекции мы рассматриваем TLS как один конкретный механизм.

Минимум криптографических понятий

Механизм	Смысл
Симметричное шифрование	Стороны используют общие секретные ключи
Согласование ключей	Стороны вырабатывают общий секрет, обмениваясь открытыми параметрами
Цифровая подпись	Владелец приватного ключа подписывает данные, публичный ключ позволяет проверить подпись

Асимметрическая криптография включает разные семейства механизмов. Не всякая пара ключей предназначена для шифрования, а подпись не следует объяснять как универсальную операцию «зашифровать приватным ключом».

В обычном полном TLS 1.3 временный обмен (EC)DHE помогает выработать общий секрет. Подпись подтверждает владение ключом аутентифицируемой стороны. Данные защищают симметричные алгоритмы с проверкой целостности, например AEAD.

Эти роли различаются. Публичный ключ из сертификата не шифрует каждый байт ответа. Закрытый ключ сервера нельзя передавать клиентам вместе с сертификатом.

PKI и центр сертификации

PKI, Public Key Infrastructure, объединяет сертификаты, участников и правила обращения с ключами. CA, Certificate Authority, проверяет основания для выпуска и подписывает сертификат.

Формулировка «CA доверяют все» неверна. Каждый клиент или его администратор выбирает доверенные корни. Хранилища доверия операционной системы, браузера, Java-приложения и отдельного инструмента могут различаться.

При выдаче domain-validated сертификата CA проверяет контроль домена в рамках установленной процедуры. Это не гарантия порядочности владельца, корректности приложения или отсутствия вредоносного содержимого сайта.

Сертификат X.509

Сертификат связывает идентичность с публичным ключом и подписью издателя.

Поле или группа	Что сообщает
Subject Alternative Name, SAN	Имена и другие идентификаторы
notBefore / notAfter	Период действия
Subject Public Key Info	Публичный ключ и его алгоритм
Issuer	Издатель сертификата
Signature	Подпись издателя
Basic Constraints, Key Usage, Extended Key Usage	Ограничения и назначение

Сертификат является публичным документом. Секретным остаётся соответствующий приватный ключ. Корневой сертификат тоже публичен, особенно защищают именно приватный ключ корневого CA.

DER содержит бинарную ASN.1-структуру. PEM добавляет текстовые метки и Base64-кодирование. Например, файл сертификата начинается строкой:

```
-----BEGIN CERTIFICATE-----
```

После неё располагаются Base64-данные, затем строка END CERTIFICATE в таком же формате. Разные PEM-метки относятся к разным объектам. Сертификат и приватный ключ не являются одним файлом по смыслу даже тогда, когда администратор хранит их в одном контейнере.

Цепочка доверия

Типичный сервер предъявляет конечный сертификат и промежуточные сертификаты. Клиент строит путь к локально доверенному корню.

Элемент	Обычная роль
Leaf	Сертификат сервиса
Intermediate CA	Подписывает leaf или другие intermediate-сертификаты
Root CA / trust anchor	Основание доверия на стороне клиента

В реальной цепочке может быть несколько intermediates, альтернативные пути и кросс-подписи. Корень не становится доверенным оттого, что сервер прислал его вместе с остальными сертификатами.

Неполная серверная цепочка иногда работает у одного клиента и не работает у другого: различаются кэши, дополнительные сертификаты и способы построения пути. Наблюдение нужно связывать с конкретным клиентом и его окружением.

Проверки клиента

Для обычного HTTPS клиент проверяет как минимум допустимый путь доверия, подписи, имена, время и ограничения использования. Успешная криптографическая проверка подписи сама по себе недостаточна.

Ожидаемое имя определяется целевым сервисом, например host из URL. Его сравнивают с подходящими идентификаторами сертификата по правилам проверки имени. Для DNS-имён используется соответствующий SAN, для IP-адреса нужен подходящий IP-идентификатор. Произвольная строка Subject или CN не заменяет эти правила.

Проверка времени зависит от часов клиента. Проверка назначения учитывает, имеет ли CA право подписывать сертификаты и подходит ли конечный сертификат для серверной аутентификации.

Проверка отзыва зависит от политики и реализации клиента. Нельзя обещать одинаковый сетевой запрос к CA при каждом HTTPS-соединении.

Жизненный цикл

1. Владелец создаёт ключ и CSR, Certificate Signing Request.
2. CA проверяет основания для выпуска и выдаёт подписанный сертификат.
3. Владелец развёртывает сертификат и нужную цепочку на сервисе.
4. До истечения срока выпускает замену и обновляет все нужные узлы.
5. При необходимости сертификат отзывают.

CSR содержит публичный ключ, запрашиваемые сведения и подпись заявителя. Приватный ключ в CSR не включается. Обновление обычно означает выпуск нового сертификата, а не изменение старого подписанного документа.

CRL, Certificate Revocation List, содержит сведения об отозванных сертификатах. OCSP, Online Certificate Status Protocol, позволяет получить статус конкретного сертификата. OCSP stapling позволяет серверу передавать соответствующий ответ клиенту. Наличие этих механизмов не означает, что все клиенты одинаково проверяют отзыв и одинаково реагируют на недоступность сервиса статуса.

Источники: RFC 5280, RFC 9525, CSR, RFC 2986, PEM, RFC 7468, OCSP, RFC 6960.

7. TLS: согласование и защищённый обмен

SNI и ALPN до начала HTTP

На одном IP-адресе могут находиться разные HTTPS-сервисы. Серверу полезно узнать имя ещё во время TLS, до получения HTTP-запроса. Для этого клиент передаёт SNI, Server Name Indication.

SNI помогает выбрать параметры сервиса, включая сертификат. Это не заменяет последующую проверку соответствия сертификата ожидаемому имени. HTTP-поле Host или псевдополе :authority относится уже к прикладному обмену.

ALPN, Application-Layer Protocol Negotiation, согласует прикладной протокол. Клиент предлагает варианты, например h2 и http/1.1, а сервер выбирает поддерживаемый. Это отдельный вопрос от выбора версии TLS.

Поэтому четыре значения могут описывать разные свойства одного соединения: имя сервиса, версия TLS, алгоритмы защиты и версия HTTP.

Обычный полный handshake TLS 1.3

Для основной модели возьмём соединение с сертификатом сервера и эфемерным согласованием ключей, без возобновления, 0-RTT и клиентского сертификата.

Направление	Сообщения	Назначение
Клиент серверу	ClientHello	Версии, алгоритмы, key share, расширения
Сервер клиенту	ServerHello	Выбранные параметры и key share
Сервер клиенту	EncryptedExtensions	Согласованные расширения, в том числе выбранный ALPN
Сервер клиенту	Certificate	Цепочка сертификатов
Сервер клиенту	CertificateVerify	Подпись, подтверждающая владение приватным ключом
Сервер клиенту	Finished	Проверка целостности handshake и знания ключевого материала
Клиент серверу	Finished	Подтверждение со стороны клиента
Стороны	Application Data	Защищённые прикладные данные

После обмена key share стороны могут вывести handshake-ключи. Поэтому последующие handshake-сообщения после ServerHello уже защищены. Клиент проверяет сертификат, подпись CertificateVerify и Finished. Наличие красивого сертификата без доказательства владения его ключом недостаточно.

Таблица упрощает реальный обмен. Например, возможен HelloRetryRequest, сервер может запросить сертификат клиента, а возобновление использует другой набор сообщений. Сервер может отправлять прикладные данные после своего Finished. Эти варианты не меняют основных ролей согласования, аутентификации и защиты данных.

Forward Secrecy

У сервера есть долгосрочный приватный ключ сертификата. Для обычного эфемерного согласования ключей создаётся временный материал конкретного соединения. Из общего секрета выводятся симметричные ключи защиты трафика по направлениям.

Forward Secrecy означает, что последующая компрометация долгосрочного ключа сама по себе не позволяет расшифровать записанные ранее сеансы, защищённые соответствующим эфемерным обменом. Это предполагает корректную реализацию и удаление временных секретов.

Не следует объяснять свойство только номером версии. TLS 1.2 может использовать ECDHE и обеспечивать Forward Secrecy. TLS 1.3 допускает режим PSK-only без DHE, для которого такое объяснение эфемерного обмена не подходит.

Историческая справка: TLS 1.2 с RSA key exchange

В старом варианте материала основным примером был следующий механизм: клиент создаёт pre-master secret, шифрует его RSA-ключом сервера и передаёт серверу. Стороны затем выводят ключи сеанса. При последующей утечке RSA private key записанный обмен может раскрыть этот секрет.

Такой механизм помогает объяснить мотивацию Forward Secrecy, но он не описывает все соединения TLS 1.2. Для TLS 1.2 существовали другие способы согласования, включая эфемерный Diffie–Hellman. В TLS 1.3 статический RSA key exchange исключён, хотя RSA-подписи могут использоваться для аутентификации.

TLS 1.0 и 1.1 устарели и не используются в учебном защищённом обмене. TLS 1.2 требует внимания к согласованным алгоритмам и настройкам. Основная модель этой лекции относится к TLS 1.3.

Граница защиты

TLS защищает обмен между двумя TLS-участниками. Если соединение заканчивается на reverse proxy, а затем прокси обращается к приложению, следующий участок рассматривается отдельно. Наличие HTTPS в браузере само по себе не описывает защиту всех внутренних соединений.

TLS не гарантирует корректность JSON, отсутствие уязвимостей приложения или права пользователя на конкретную операцию. После расшифрования данные доступны процессу и могут попасть в память, логи или хранилище.

Наблюдателю сети остаются доступны IP-адреса, объём и время передачи. Обычный ClientHello может раскрывать SNI. Механизм ECH предназначен для защиты ClientHello, поэтому утверждение «имя всегда открыто в TLS» тоже слишком категорично. Наличие ECH зависит от поддержки и использования сторонами.

Диагностика TLS

Наблюдение	Что следует проверить
Сертификат просрочен	Период действия и часы клиента
Имя не совпало	URL, SNI, SAN предъявленного сертификата
Нет доверенного пути	Серверную цепочку и trust store клиента
Нет общей версии/алгоритмов	Возможности сторон и точный TLS alert
Общий handshake failure	Контекст: параметры, клиентский сертификат, серверные журналы

Текст ошибки зависит от библиотеки. Он не всегда сразу устанавливает, какая сторона настроена неверно. Например, ошибка построения цепочки может быть связана как с отсутствующим intermediate на сервере, так и с намеренно ограниченным trust store клиента.

Не нужно превращать curl -k или отключение certificate verification в исправление. Это меняет условия проверки и убирает важную гарантию соединения. Для учебного CA задают отдельный файл доверия конкретному клиенту, не устанавливая его без необходимости в системное хранилище.

Команды ниже требуют отдельно подготовленного учебного сервиса, файла course-ca.pem и разрешения имени api.course.test:

```
curl --cacert course-ca.pem -v https://api.course.test/
```

В Windows удобнее явно вызывать curl.exe, чтобы избежать совпадений с оболочечными командами. Подробный вывод может содержать секретные заголовки, поэтому его очищают перед передачей.

В Linux/WSL с OpenSSL можно проверить серверное соединение:

```
openssl s_client -connect api.course.test:443 \
  -servername api.course.test \
  -verify_hostname api.course.test \
  -CAfile course-ca.pem -verify_return_error \
  -alpn h2,http/1.1 -showcerts
```

Здесь servername задаёт SNI, verify_hostname включает проверку ожидаемого имени, CAfile задаёт учебный trust anchor. Один showcerts лишь показывает сертификаты и сам по себе не доказывает доверие. Наличие OpenSSL на каждом студенческом ноутбуке пока не предполагается.

Источники: TLS 1.3, RFC 8446, TLS 1.2, RFC 5246, SNI, RFC 6066, ALPN, RFC 7301, отказ от TLS 1.0/1.1, RFC 8996, ECH, RFC 9849.

8. HTTP/1.1, HTTP/2 и HTTP/3

Что сохраняется

Методы, статусы и семантика полей HTTP остаются общими. GET продолжает обозначать запрос представления ресурса, а Content-Type продолжает описывать содержимое. Версии различаются устройством передачи сообщений.

Текстовый JSON можно передавать и через HTTP/1.1, и через HTTP/2, и через HTTP/3. Бинарный формат фреймов HTTP/2 не превращает автоматически тело JSON в Protobuf.

Свойство	HTTP/1.1	HTTP/2	HTTP/3
Заголовки и организация сообщений	Стартовая строка, текстовые поля	Бинарные фреймы	Бинарные фреймы
Независимые обмены	Ответы по порядку внутри соединения	Потоки в общем TCP-соединении	Потоки QUIC
Сжатие полей	Отдельного HPACK/QPACK нет	HPACK	QPACK
Обычный HTTPS-стек	TLS поверх TCP	TLS поверх TCP	QUIC с TLS 1.3 поверх UDP

Строка про защиту относится именно к HTTPS. HTTP/1.1 и HTTP/2 имеют также варианты работы без TLS. Для HTTP/3 шифрование является частью используемого QUIC.

В HTTP/1.1 возможно отправить несколько запросов без ожидания каждого ответа, pipelining, но ответы внутри такого обмена должны идти в соответствующем порядке. Для параллельного обслуживания браузеры традиционно использовали несколько соединений.

HTTP/2: потоки и фреймы

HTTP/2 выделяет отдельные потоки, streams, для обменов. Сообщения передаются фреймами, которые содержат тип и идентификатор потока. Соединение может чередовать фреймы разных потоков.

Это позволяет отправлять части разных ответов, не дожидаясь полного завершения одного длинного ответа. Но все байты по-прежнему идут через один упорядоченный TCP-поток. Потеря участка TCP может задержать выдачу последующих байтов приложению, в том числе относящихся к другим HTTP/2-потокам.

Здесь достаточно различить блокировку порядка прикладных ответов и блокировку доставки байтов транспорта. Почему TCP ведёт себя именно так, разберём на третьей лекции.

HPACK сжимает поля HTTP/2. Это другая задача, чем gzip содержимого. Приоритеты выражают предпочтения обслуживания, но не дают приложению абсолютной гарантии порядка или времени исполнения.

HTTP/2 также предусмотрел Server Push: сервер мог заранее отправить ресурс без отдельного запроса на него. Механизм полезно знать как часть стандарта и истории оптимизаций, но не следует подавать как универсальную современную браузерную практику. Chrome отключил поддержку по умолчанию с версии 106.

HTTP/3 и QUIC

QUIC предоставляет надёжные потоки и другие транспортные функции поверх UDP. UDP сам по себе не добавляет ни надёжность, ни независимое восстановление потоков.

QUIC использует TLS 1.3 handshake для согласования ключей и аутентификации. При этом он не переносит данные в TLS records поверх UDP: защиту пакетов организует сам QUIC. Поэтому схема «HTTP → TLS record layer → TCP или QUIC» неверна.

Потоки QUIC позволяют избежать обязательного ожидания потерянных байтов одного потока перед выдачей данных другого потока. Это не полная независимость: потоки делят сетевые ресурсы и управление перегрузкой, а на прикладном уровне возможны зависимости.

В этом занятии нам нужна общая карта. Подробный блок сохранён для четвёртой лекции после изучения транспорта:

Тема лекции 4	Что предстоит объяснить
Надёжность QUIC	Доставка данных, подтверждения, восстановление
Потоки и блокировки	Какие зависимости исчезают и какие остаются
Handshake	1-RTT, возобновление, ограничения 0-RTT и повторная передача ранних данных
Connection ID	Идентичность соединения и смена сетевого пути
Управление потоком и перегрузкой	Почему общие ограничения влияют на несколько потоков
HTTP/3	Отображение HTTP на QUIC и роль QPACK
Обнаружение и fallback	Alt-Svc, HTTPS DNS records, доступность UDP

Для 0-RTT особенно важно обсуждать возможность replay и ограничения безопасного применения, а не только экономию времени.

Клиент может узнать о поддержке HTTP/3 через Alt-Svc, DNS-запись HTTPS или ранее сохранённую информацию. Поэтому утверждение «первый запрос всегда идёт по HTTP/2, затем всё переключается» не является универсальным правилом. При недоступности QUIC возможен fallback. Алгоритм и тайминги зависят от клиента.

Источники: HTTP Semantics, RFC 9110, HTTP/2, RFC 9113, HTTP/3, RFC 9114, QUIC, RFC 9000, TLS в QUIC, RFC 9001, Server Push в Chrome.

9. Самопроверка и диагностический отчёт

Задачи

1. Письмо содержит Content-Type: text/plain; charset=UTF-8 и Content-Transfer-Encoding: base64. В каком порядке обработать тело? Что произойдёт, если пропустить первый шаг?

2. Письмо содержит plain text и HTML в multipart/alternative. Следует ли показать их как два независимых вложения? Как изменился бы смысл multipart/mixed?

3. Клиент получил Content-Encoding: gzip и Content-Length: 120. После распаковки получилось 800 байтов. Доказывает ли это ошибку длины?

4. Сертификат имеет корректную подпись доверенного CA, но клиент сообщает hostname mismatch. Какие значения нужно сравнить?
5. Парсер сообщил ошибку JSON. Доказано ли, что тело HTTP уже полностью получено?
6. Один клиент принимает сертификат, другой отвергает цепочку. Какие различия окружения стоит проверить до вывода «сломана сеть»?
7. HTTP/2 передаёт фреймы в бинарном формате. Следует ли из этого, что application/json больше не используется?
8. Почему нельзя объяснить преимущества QUIC только фразой «он работает по UDP»?

Краткие ориентиры для проверки ответа

1. Сначала Base64, затем UTF-8. Без первого шага останется строка кодирования.
2. Alternative задаёт версии одного содержимого. Mixed объединяет разные части.
3. Нет. Content-Length может относиться к сжатому содержимому. Важно определить, какие байты измерили.
4. Ожидаемое имя из обращения, SNI для выбора сервиса и SAN сертификата. SNI и проверка имени решают разные задачи.
5. Нет. Парсер может работать потоково. Нужно проверить framing и фактически полученные байты.
6. Trust store, intermediates, кэши, настройки построения пути, версии библиотек и конкретный узел назначения.
7. Нет. Формат обрамления HTTP и формат тела независимы.
8. Надёжность, потоки, защита и управление перегрузкой реализованы в QUIC, а не предоставлены UDP.

Что передать коллеге

Хороший диагностический отчёт содержит достаточно данных, чтобы отделить наблюдения от предположений:

- Когда и из какого окружения выполнялась проверка.
- К какому имени, адресу и порту обращался клиент.
- Какие версии TLS и HTTP согласованы, если этот этап состоялся.
- Какой запрос отправлен и какие существенные поля получены.
- Какой объём и на какой стадии обработки зафиксирован.
- Точный текст ошибки и достигнутый этап.
- Какую гипотезу проверяли и чем отличался повторный запуск.

Секреты и персональные данные в отчёт не включают. Фраза «не работает JSON» становится полезнее, если заменить её наблюдением: «получен статус 200, Content-Type text/html, полное тело начинается с формы входа». Это уже позволяет предметно выбирать следующую проверку.

Главная мысль двух первых лекций: сеть нужно рассматривать как последовательность взаимодействий и преобразований. Тогда вместо общего обвинения сети появляются конкретный этап, подтверждённый факт и следующий диагностический шаг.

10. Дополнительные источники

Помимо ссылок в разделах, для справочного чтения полезны:

- Реестр media types IANA.
- Base64 и другие Base-N кодирования, RFC 4648.
- SMTP 8BITMIME, RFC 6152.
- BINARYMIME, RFC 3030.
- SMTPUTF8, RFC 6531.
- Unicode Standard.
- XML 1.0.
- RADIUS, RFC 2865.
- Diameter AVP, RFC 6733.
- Каталог спецификаций 3GPP: MAP TS 29.002, CAP TS 29.078, RRC TS 36.331 и 38.331, NAS TS 24.301 и 24.501.
- Alt-Svc, RFC 7838.
- SVCB и HTTPS DNS records, RFC 9460.
- curl: руководство.
- OpenSSL s_client.